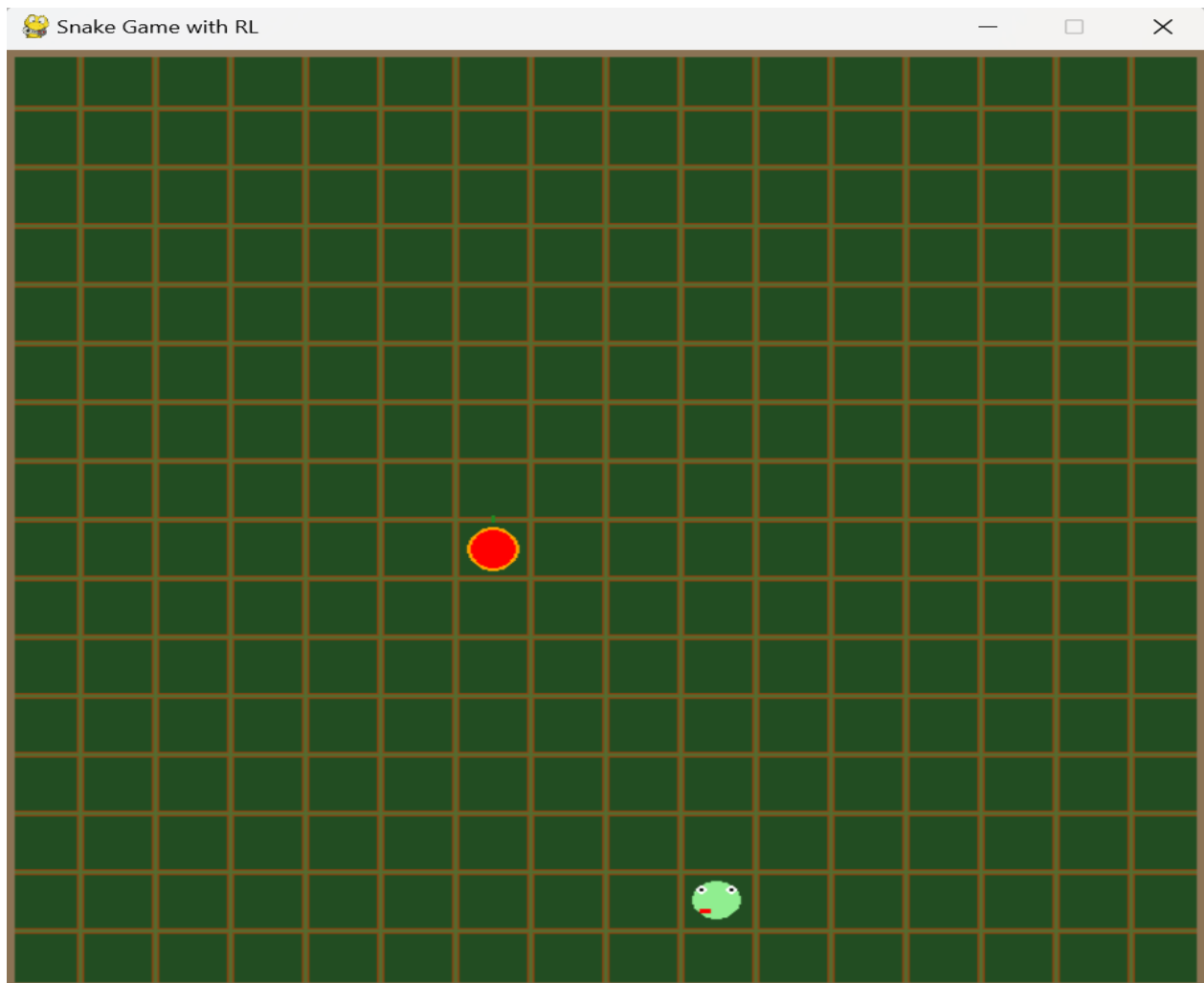


Intelligent Agents – Final Project (Ketan Khanna)

Project Topic: The Agent learning to play the game **Snake** using Reinforcement Learning

Objective: For the agent to navigate the 16x16 grid like structure in search of one cell that contains food (the goal). The agent is reflected as a snake and navigates through individual cells in search of the food. However, the agent must not go outside the grid or encounter any boundary, otherwise it will be considered “Game over” and that session/episode will result in a loss. Upon encountering any food, the agent will proceed to the next level at which point it will have to navigate a faster paced environment (due to the incremental difficulty arising from a fixed rise in Frames Rates per Second). Additionally, with the consumption of one cell of good the snake/agent will consequently elongate by one cell as a result (considered its neck). The agent will win the game upon completion of 8 levels.



(Diagram 1: Snapshot of the agent trying to find the food)

Techniques: The model I employed was built with the implementation of reinforcement learning so the agent could adapt to its environment and successfully level up and complete the game. Q-learning principles combined with deep neural networks were applied to train the agent over multiple episodes on how to optimally find food and be rewarded for its search. I also added penalization for hitting the boundaries as I noticed that agent would be quite focused on exploring these uncharted and “dangerous” areas on the grid.

Functions employed:

- 1) *epsilon-greedy* was a function that was implemented within the code to reinforce the agent in exploiting its learned policy for it to better succeed in its objective over time. The agent received positive rewards (+5000) for finding food and experienced negative reinforcement (-1000) when hitting a boundary or its own tail. Though through my observation it would take hours or almost days of the code running for it to succeed in its task as the increase in proficiency was nominal.
- 2) *reset_game()* was employed to reset the game to its initial configurations in the case of a loss or game over. Additionally, we used this function to ensure the agent always starts in the center of the grid, the food location is consistently reset after every episode.
- 3) *place_food* was also configured to ensure the food is placed on the grid without encountering the snake's body upon the game starting. Additionally, the code was instructed to not have the food placed on any existing “danger/kryptonite” zones outside the grid. To challenge the snake, food is placed randomly away from the position in the previous episode or where it was last eaten.
- 4) *move_snake* had the purpose of computing the new head position of the snake while preventing it from reversing into its own tail. This was a continuous challenge in the beginning and after continuous troubleshoots I realized that “game over” was occurring due to the code not accounting for this.
- 5) *check_collision_type* identifies the type of collision taking place during the snake/agent's journey and by using this code the system can penalize the agent accordingly.
 - "food" (rewarded).
 - "boundary" (penalized).
 - "tail" (penalized).
- 6) *get_grid_state* returns a grid like structure of the current game. It also marks on the board the position of the snake's body and the location of the food. It allows for the neural network to provide input to the agent and to observe its immediate vicinity for effective decision making.

- 7) *Choose_action* implements the epsilon-greedy code for the agent to proceed with a particular action when navigating its environment. The snake initially begins as a naïve agent and explores random actions to better learn about its surroundings. The exploits and knowledge gained through Q learning reinforces steps the agent should take to better find food and complete its objective. I decided to remove rewards for completing a level since it just requires 1 food to do so.
- 8) *replay_and_train* train the agent using relevant experiences gained through Q learning from previous episodes and updates the neural network based on observed rewards and future expected rewards. It reinforces the agent to search for the food diligently (positive behaviours) and discourages the agent from encountering a boundary or its own tail (negative reinforcement).
- 9) *update_target_network()* was used to copy the weights of the main model to the target model after a certain number of episodes. This stabilized learning and ensured the Q values would remain consistent for some time, ensuring proper agent learning.
- 10) *draw_game* was used to create the virtual jungle environment in which the agent would navigate to find food.
- 11) *run_snake_game()* s coordinates all components of the digital game environment to ensure that the agent continues to gain proficiency in terms of:
 - a) Managing its learning using previous functions
 - b) Perform best course of action
 - c) Rewards and Penalties

Engineering Challenges

- 1) **Grid Size:** I consistently configured the grid size to ensure that it was suitable to train the agent. Not big enough to completely confuse the agent but not small enough that it would not gain enough learning. I settled on a 16x16 grid which I found to have a necessary enough size to accommodate the agent's learning while also being small enough that it would not become lost, confused or unstable.
- 2) **Tail Reversal and avoiding back-tracking:** This was one I did not immediately notice though gaps in my code in which the agent's head kept reversing into its own neck kept causing a game over. To remedy this, I added code that would cause the head to only move forward and that a collision with itself would only occur when moving into its own tail. However, I will mention this almost never happened as the agent never became intelligent enough to get to such a level. The agent was also trained not to continuously revisit cells in which it had not found food.
- 3) **Food appearing next to boundary:** After careful observation, I had to modify the code to ensure that food never appeared immediately next to the boundary as it was too challenging for the agent to solve within the timeframe of a few hours to a day. Therefore, I modified the code for the food to appear in a further position away from the boundary upon each new episode.
- 4) **The agent lacking proficiency:** Though I wouldn't consider my agent a failure, its discouraging not seeing it succeed as much as I wanted within the desired timespan. I was really expecting the Q learning logic to be implemented at a quicker level and noticed only marginal improvements in the agent over time (max it achieved was level 3 or 4) through multiple iterations of training and different code. After so many iterations I'm still unsure if I moved backwards at one point in terms of how the agent was trained and if adjustments to code just added further lack of performance. Additionally, I expected the agent to learn the danger zones much better than it did. The agent kept revisiting them even when reducing the level of exploration. Nevertheless, the agent did seem to perform better at finding the food after a substantial number of episodes (multiple thousands capped at 25k).
- 5) **Level Progression Logic:** The level progression was also breaking in between and code required further adjustment to function properly. It would continuously stop at level 2 or encounter food and reset to level one. At times configuration felt like a nightmare.

```

Collected food! Remaining: 0
Level up! Now on level 2.
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 18ms/step
Game Over - Hit boundary at (9, -1). Ignoring out-of-bounds
=== Episode 68/25000 - Level 2 (FPS: 10, Food Needed: 1) ==
Game Over. Resetting game...
=== Episode 69/25000 - Level 1 (FPS: 8, Food Needed: 1) ===
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
Collected food! Remaining: 0
Level up! Now on level 2.
1/1 [=====] - 0s 21ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 16ms/step
1/1 [=====] - 0s 16ms/step
Game Over - Hit boundary at (16, 4). Ignoring out-of-bounds
=== Episode 70/25000 - Level 2 (FPS: 10, Food Needed: 1) ==
Game Over. Resetting game...
=== Episode 71/25000 - Level 1 (FPS: 8, Food Needed: 1) ===
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step
1/1 [=====] - 0s 15ms/step

```

Diagram 2: Python Output of Level Progression

- 6) **Training Stability:** Adjusting rewards and penalization rates for certain actions also required continuous tinkering to ensure the agent was learning proficiently. However, without a team or multiple people working a project like this I would not be inclined to enhance it further without knowing where the deficiencies lay.
- 7) **Difficulty scaling:** Finding the correct FPS to ensure agent learning also proved to be difficult as the agent needed enough time to effectively make decisions while also learning from its immediate environment. The FPS was continuously tinkered with to ensure the agent was performing at a fast enough rate with enough difficult while also being visually understood.

- 8) **Visual Enhancements:** A personal desire of mine was to have the simulation play in a aesthetically pleasing manner. However, this may have interfered with the snake's performance with respect to resource optimization. In the future, it is probably better to focus on just having an output of results rather than an entire snake game play out in almost real time. However, the jungle like look of the grid was pleasing to the eye. I also did not want to pursue a simulation strategy due to potential overfitting or lack of visual representation.

Conclusion

In conclusion, the snake game was an exciting project to tackle due to the complexity of the agent's objective and the nature of our findings. The agent was reluctant to learn quickly and initially was more concerned with navigating its environment to learn of any dangers. Upon gaining proficiency, the agent was still relegated to at most achieving level 4 or 5 with the different code iterations I created, not even allowing scenarios in which it could encounter its own tail. The numerous challenges surrounding the grid logic (finding the proper placement for food) and agent naivety (unknown of its surrounding environment or reversing into its own tail) caused many frustrating moments. Nevertheless, the agent over time did seem to adapt better to its immediate environment and did seem to gain some proficiency over multiple episodes, though this was nominal and took a substantial amount of time. Though I am convinced further enhancements would have improved the agent's performance, I was still expecting better when employing reinforcement learning or agent manipulation. This project was invaluable in terms of how learning how deep learning and intelligent agents can maintain a synergetic relationship with respect to enhancing real world scenarios and environments.